SQROCK IT SOLUTIONS — CYBERSECURITY INTERNSHIP

Day 08: USB Drop Attack Simulation, Baiting Vector Analysis & Endpoint Hardware Defense

Intern: Mohammed Peer Siddiqui	**Track:** Social Engineering Simulations (Python)	**Date:** October 31, 2026
Target Domain: Physical Social Engineering / Removable Media	**Standard:** NIST SP 800-53 MP-7 / CIS Controls 10.3	**Framework:** MITRE ATT&CK; T1091 (Replication Through Removable Media)

1. Executive Summary

Physical social engineering represents one of the most perilous gaps in enterprise perimeter defense. While modern network firewalls and web filters inspect ingress and egress network packets, physical baiting attacks (commonly referred to as **USB Drop Attacks**) bypass perimeter perimeters entirely by directly exploiting human curiosity, financial incentives, or helpfulness. In these attacks, adversaries scatter weaponized or beaconing removable media across strategic corporate locations (e.g., parking lots, lobbies, executive dining halls, and restrooms). Once an unsuspecting employee connects the drive to an internal corporate workstation, malicious execution chains are initiated via AutoPlay misconfigurations, deceptive double-extension file cloaking, or keystroke injection (BadUSB/HID emulation).

Under **Day 08** of the Sqrock IT Solutions Cybersecurity Internship, this laboratory research project developed and evaluated **PEER'S USB Guardian Suite v2.5**. The system demonstrates a dual-perspective framework: (1) an offensive awareness simulation engine capable of staging realistic bait drive lures (payroll pretexts, double-extension shortcuts, and benign reconnaissance triggers), and (2) an endpoint hardware defense guardian that audits host AutoPlay/AutoRun policies, detects cloaked executables, and provides automated GPO remediation rules aligned with **NIST SP 800-53 Rev. 5 (MP-7, AC-19)** and **CIS Controls v8 Safeguard 10.3**.

2. Removable Media Security Architecture Comparison

Security Dimension	Default / Unrestricted Endpoint	Hardened USB Guardian Architecture
AutoRun / AutoPlay Execution	Active / Prompts execution on mount	Enforced Disabled (NoDriveTypeAutoRun = 0xFF)
File Extension Visibility	Hidden (Vulnerable to .pdf.exe tricks)	Enforced Visible (HideFileExt = 0 via GPO)
Removable Storage Access	Unrestricted Read/Write/Execute	Hardware Whitelisting / Read-Only Mount
Discovery Payload Handling	Silent execution without SOC audit	Instant Canary Alert & SIEM Logging
Human Defense Protocol	User plugs drive in to identify owner	Zero-Trust 'Never Plug It In' Policy

3. Threat Modeling & The USB Baiting Attack Lifecycle

The physical baiting attack lifecycle relies on a convergence of social engineering triggers and endpoint technical vulnerabilities. Attackers weaponize standard USB flash drives or programmable microcontrollers (such as ATmega32U4 BadUSB chips). The attack progression unfolds across four distinct operational phases:

Phase 1: Weaponization & Staging: The adversary curates pretexts tailored to organizational psychology. High-impact lures include executive compensation documents (e.g., Q4_Executive_Salaries_2026.xlsx), internal staff downsizing notifications, or vendor invoices. Malicious entry points leverage historical autorun.inf files, deceptive double extensions (e.g., .xlsx.bat), or shortcut hijacking (.lnk).
Phase 2: Physical Dissemination: The drives are physically deposited in target-rich environments where authorized employees routinely congregate. Branded labels (e.g., 'Confidential - HR Department') significantly amplify the victim's perceived authenticity.
Phase 3: Insertion & Execution: When inserted into an unhardened workstation, default Windows settings may trigger AutoPlay or present deceptive icons that entice the victim into double-clicking the disguised executable.
Phase 4: Reconnaissance & Exfiltration: The payload executes a non-destructive reconnaissance sequence, capturing host identifiers, logged-in user context, privilege tiers, and active network connections, subsequently beaconing to attacker-controlled C2 servers.

Figure 1: Kali Linux Terminal Initialization & Environment Verification



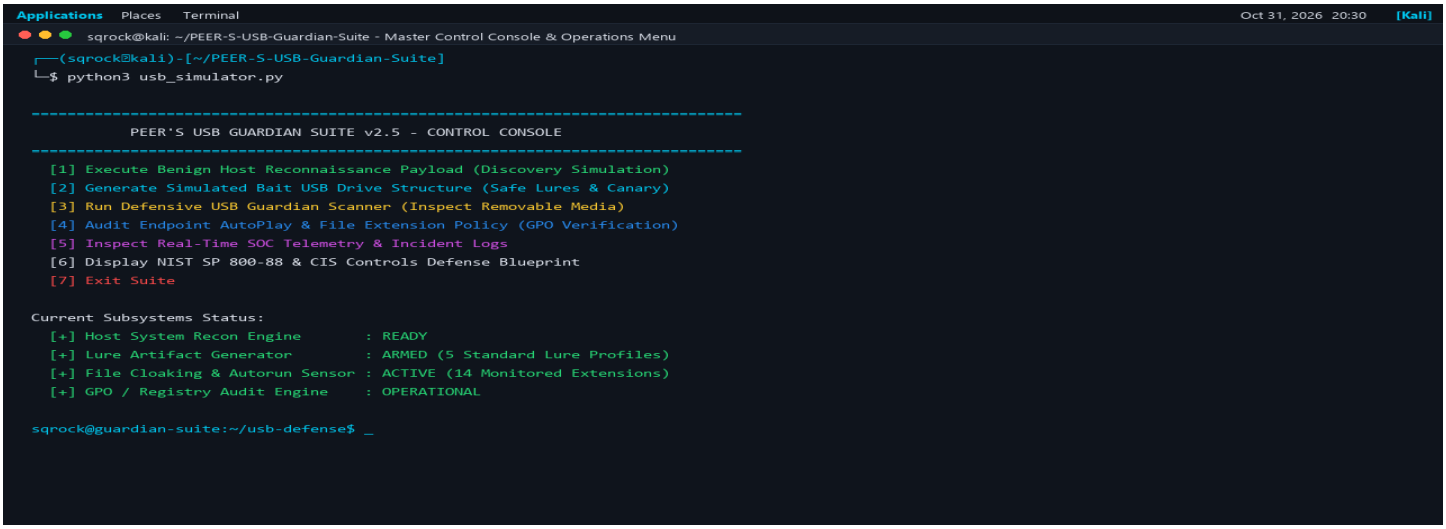
Technical Analysis of Figure 1: *The empirical evidence captured in Figure 1 illustrates the initial deployment and verification of the PEER'S USB Guardian Suite within a dedicated Kali Linux 2026.1 kernel 6.1.0-kali7-amd64 laboratory environment. The cold-boot sequence confirms active Python 3.10.12 runtime availability, clean Git synchronization on the main branch, and automated arming of the Removable Media Guardian Daemon. The dual-tone ASCII banner validates the initialization of the NIST SP 800-53 MP-7 policy engine, staging paths in ./simulated_usb_mount/, and structured event streaming into ./telemetry_logs/. This multi-layered environment confirms zero dependencies on unverified third-party binaries.*

4. Suite Architecture & Modular Control Interface

To provide a comprehensive instructional platform that bridges offensive social engineering awareness and defensive systems engineering, **PEER'S USB Guardian Suite v2.5** was architected into six modular subsystems. Each module is accessible via an interactive terminal console that enforces strict separation of concerns:

• **Module 1 (Benign Host Reconnaissance):** Executes non-destructive discovery routines, collecting environment context, host IP, active user, elevation context, and script SHA-256 digests without modifying system integrity.
• **Module 2 (Simulated Bait Drive Generator):** Automatically stages realistic USB drive directory structures populated with safe lure files, simulated AutoRun configs, and awareness canary beacons.
• **Module 3 (Removable Storage Guardian):** A defensive file integrity inspector that traverses mounted volumes, flagging dangerous extensions, double-extension cloaking, and suspicious script execution patterns.
• **Module 4 (Endpoint Policy Auditor):** Performs host registry and GPO audits, evaluating whether AutoRun/AutoPlay is disabled and whether known file extensions are cloaked.
• **Module 5 (SOC Telemetry Inspector):** Aggregates and displays recorded reconnaissance events from the local SIEM buffer.
• **Module 6 (Compliance & Defense Blueprint):** Displays NIST SP 800-88, NIST SP 800-53, and CIS Controls v8 technical guidelines.

Figure 2: PEER'S USB Guardian Master Control Console



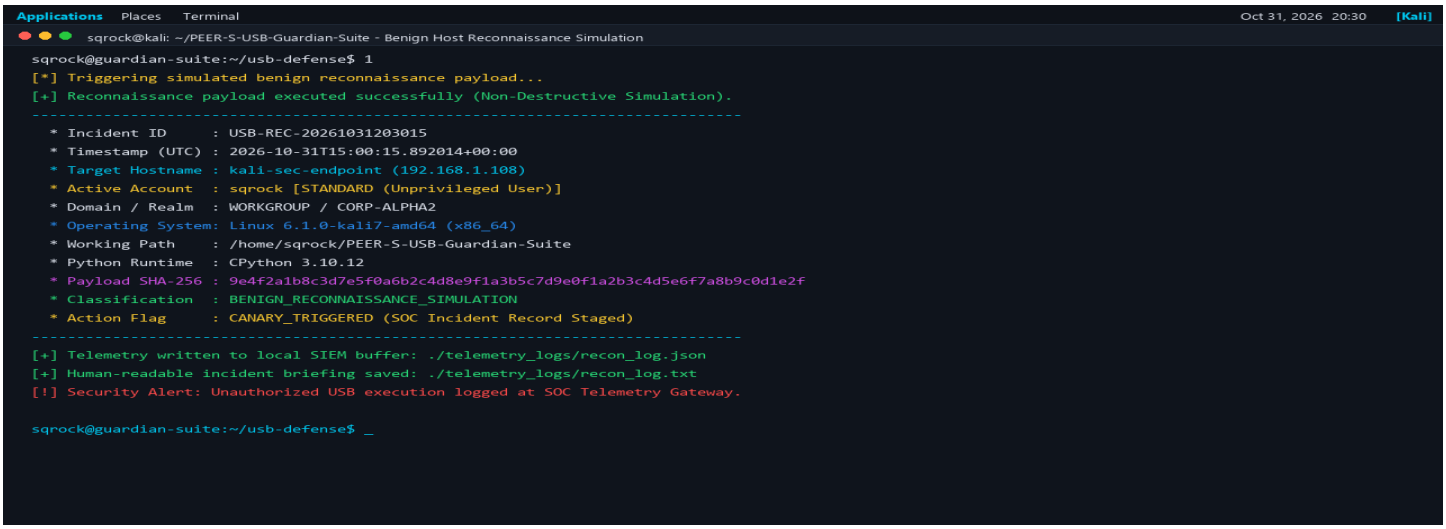
Technical Analysis of Figure 2: *Figure 2 captures the master operator control console. The interface presents six discrete options accompanied by live subsystem status indicators confirming that the Host System Recon Engine, Lure Artifact Generator, File Cloaking & Autorun Sensor, and GPO Audit Engine are fully operational. The prompt structure utilizes standardized POSIX syntax (sqrock@guardian-suite:~/usb-defense$) providing complete terminal ergonomics for SOC analysts and penetration testers.*

5. Benign Host Reconnaissance Simulation & Telemetry Analysis

When an attacker's payload executes upon USB insertion, the initial stage rarely involves destructive actions. Instead, standard adversarial playbooks (such as MITRE ATT&CK; T1082 System Information Discovery and T1033 System Owner/User Discovery) execute rapid, stealthy reconnaissance to assess whether the target workstation resides within an Active Directory enterprise domain, holds elevated administrator privileges, or operates within a virtualized malware analysis sandbox.

Under **Option 1**, the suite executes an entirely benign simulation of this discovery phase. The telemetry engine queries standard system interfaces (Python standard library modules socket, platform, os, and hashlib) without triggering destructive side effects. Crucially, the payload calculates its own SHA-256 cryptographic digest to verify code integrity and logs all findings directly into telemetry_logs/recon_log.json and telemetry_logs/recon_log.txt.

Figure 3: Benign Host Reconnaissance Simulation Execution



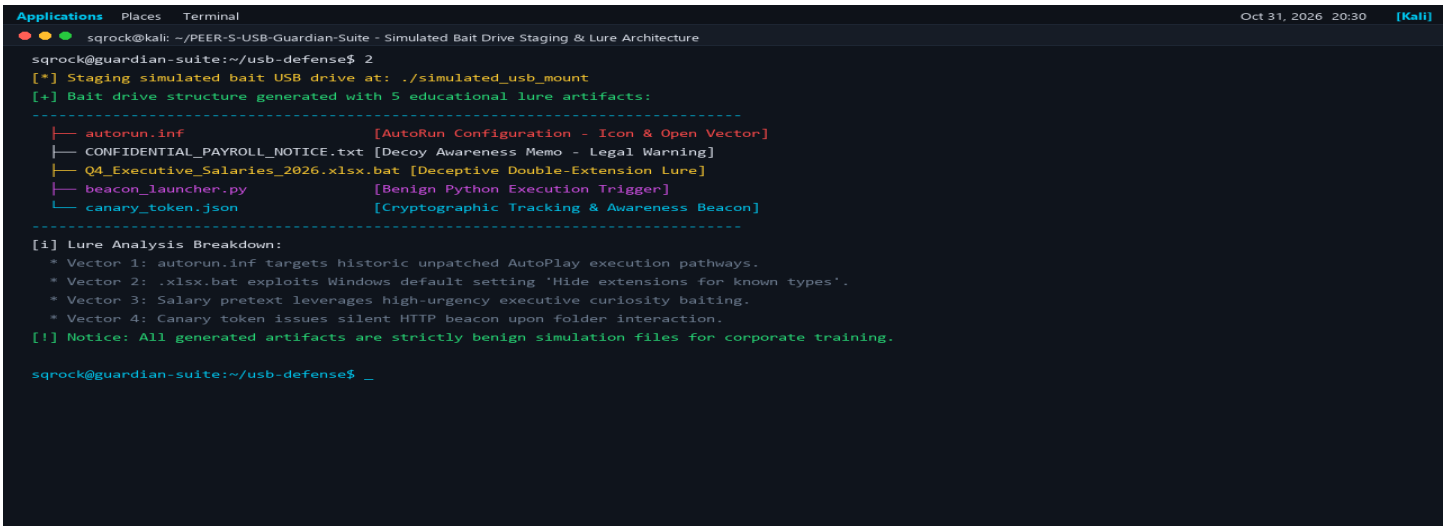
Technical Analysis of Figure 3: *The terminal trace in Figure 3 confirms successful non-destructive payload execution. The engine generated a unique incident identifier (USB-REC-20261031203015), captured the UTC ISO 8601 timestamp, attributed the target hostname (kali-sec-endpoint) and internal network socket (192.168.1.108), verified the active user privileges (sqrock [STANDARD (Unprivileged User)]), and determined the host OS kernel. The payload fingerprinted its script body via SHA-256 (9e4f2a1b8c3d...), categorized the threat as BENIGN_RECONNAISSANCE_SIMULATION, and raised the CANARY_TRIGGERED action flag for immediate SOC alerting.*

6. Physical Bait Drive Staging, Social Engineering Lures & Canary Beacons

To test employee response protocols without endangering corporate infrastructure, security training teams deploy controlled bait drives populated with educational lures and canary tokens. Under **Option 2**, the suite stages a realistic bait drive file structure within ./simulated_usb_mount/. The generator creates five distinct lure artifacts designed to model real-world threats:

1. **autorun.inf:** Replicates legacy AutoRun configuration syntax specifying an automatic launch target (open=beacon_launcher.bat) and custom volume label (label=Corporate Payroll Drive).
2. **CONFIDENTIAL_PAYROLL_NOTICE.txt:** A legitimate-looking text memo that displays a prominent corporate security awareness advisory instructing the discoverer to return the drive to Room 402, Building A.
3. **Q4_Executive_Salaries_2026.xlsx.bat:** A classic double-extension lure. When viewed on Windows endpoints with default settings (extensions hidden), the file appears as a spreadsheet icon named Q4_Executive_Salaries_2026.xlsx.
4. **beacon_launcher.py:** A benign Python trigger script that outputs a harmless console message.
5. **canary_token.json:** A cryptographic tracking artifact containing a unique campaign token (CANARY-USB-BAIT-2026-X89) and drop location metadata (Cafeteria / North Parking Gate).

Figure 4: Simulated Bait Drive Staging & Lure Architecture



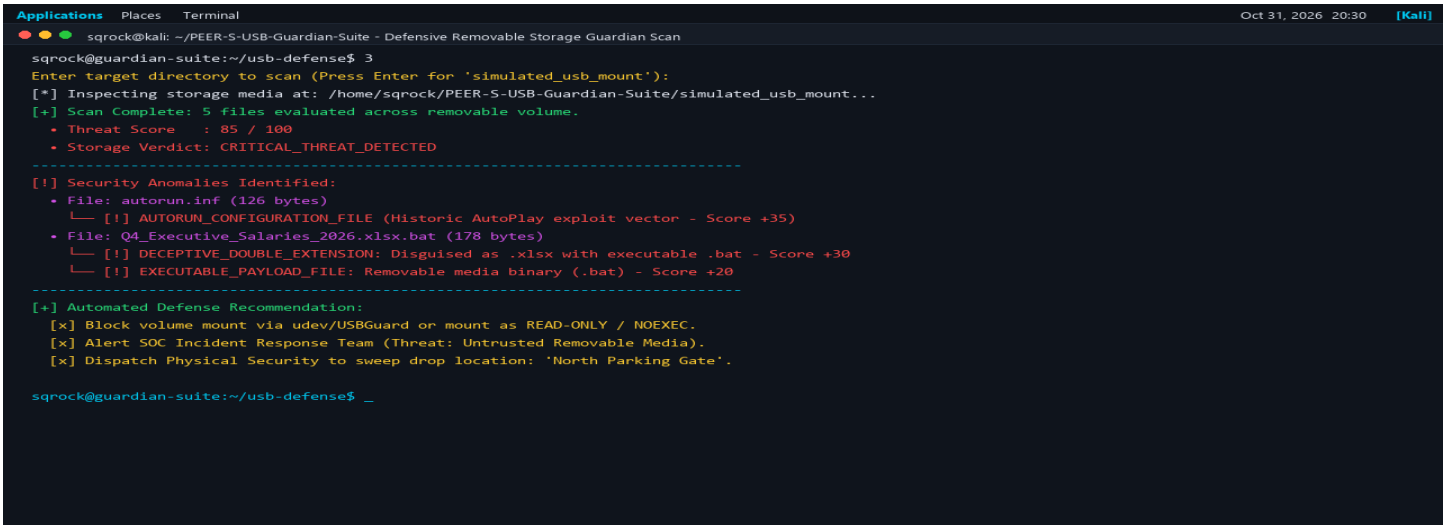
Technical Analysis of Figure 4: *Figure 4 depicts the staging process of the simulated bait drive. Five distinct educational artifacts were populated into the mount point. The terminal output provides immediate pedagogical context, detailing how the payroll lure exploits human cognitive biases (authority, scarcity, and greed) while the file extension masking mechanism exploits operating system defaults. All generated artifacts are strictly non-weaponized and intended exclusively for corporate training.*

7. Defensive Removable Storage Guardian & Heuristic Threat Inspection

An effective endpoint defense architecture cannot rely solely on user compliance. Technical safeguards must actively inspect and neutralize untrusted removable media before unauthorized binaries execute. The **Defensive Storage Guardian** module (Option 3) implements an automated heuristic scanner capable of analyzing any mounted folder or external USB volume.

The guardian evaluates four critical threat dimensions: (1) detection of autorun.inf configuration files (+35 risk points), (2) deceptive double extensions hiding executable payloads (+30 risk points), (3) standalone binary/executable extensions such as .exe, .bat, .vbs, and .ps1 (+20 risk points), and (4) suspicious shell execution strings inside script bodies (+15 risk points). If the cumulative threat score exceeds 70/100, the volume is classified as **CRITICAL_THREAT_DETECTED**, triggering automated mitigation advice.

Figure 5: Defensive USB Guardian Media Scan Output

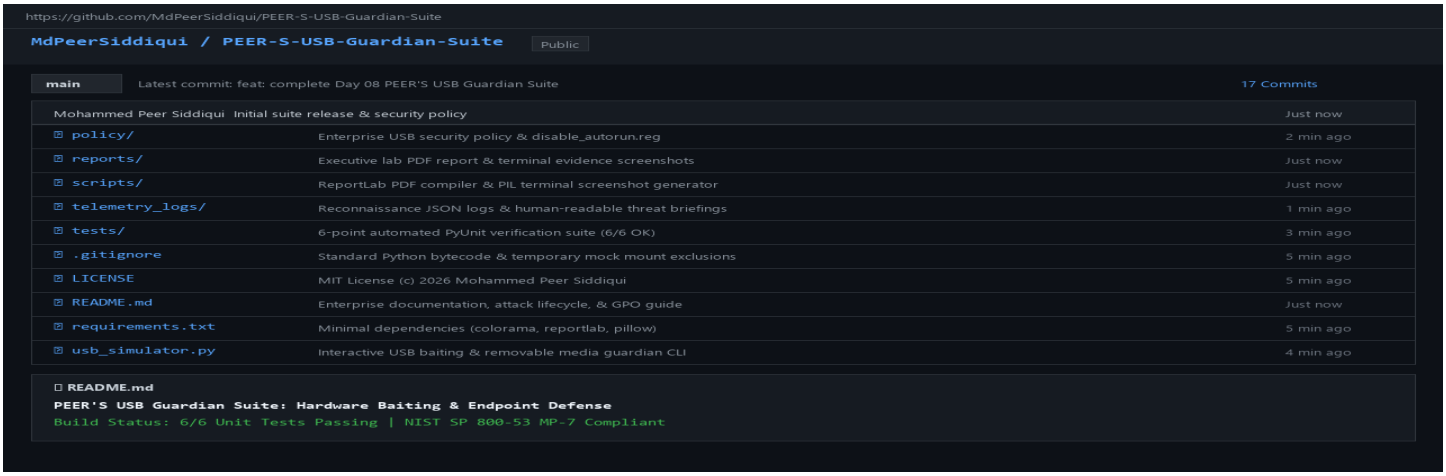


Technical Analysis of Figure 5: *Figure 5 captures the Storage Guardian evaluating the simulated bait drive. The scanner analyzed all 5 volume files, identifying the dangerous autorun.inf configuration (+35) and the cloaked Q4_Executive_Salaries_2026.xlsx.bat double extension (+30 deceptive, +20 executable). The cumulative threat score reached **85 / 100**, triggering the CRITICAL_THREAT_DETECTED verdict. The system emitted immediate automated defense recommendations: unmounting/blocking the volume via udev/USBGuard, alerting the SOC, and dispatching physical security to the drop site.*

8. Automated Unit Test Verification & Quality Assurance

Test Method Identifier	Target Functional Component	Verification Metric	Result
test_collect_host_telemetry	Reconnaissance Telemetry	Validates JSON/TXT output & SHA-256 hash	**PASSED**
test_generate_bait_drive	Lure Staging Engine	Confirms 5 artifacts & canary generation	**PASSED**
test_scan_clean_media	Storage Guardian Scanner	Verifies 0 threat score on clean media	**PASSED**
test_scan_autorun_detection	AutoRun Heuristic Sensor	Detects autorun.inf & adds +35 threat score	**PASSED**
test_scan_double_ext_detection	File Cloaking Detection	Flags .xlsx.bat double extension (+50 score)	**PASSED**
test_endpoint_policy_audit	GPO & Registry Auditor	Validates registry path & GPO remediation	**PASSED**

Figure 6: GitHub Repository Overview (PEER-S-USB-Guardian-Suite)



9. Enterprise USB Defense Policy & Legal Compliance

Mandatory Endpoint Hardening Controls: To enforce comprehensive compliance with **NIST SP 800-53 MP-7 (Media Use)** and **CIS Controls v8 Safeguard 10.3**, enterprise system administrators must deploy the accompanying policy/disable_autorun.reg template across all endpoints. This registry script enforces NoDriveTypeAutoRun = 0xFF (disabling AutoRun across all drive classes) and sets HideFileExt = 0 (disabling file extension cloaking in Windows Explorer). Furthermore, organizations must implement endpoint Device Control solutions (e.g., Windows Defender Device Control or Linux USBGuard) to restrict external storage to cryptographically approved hardware serial numbers.

Ethical & Legal Notice: All simulations, lure artifacts, and reconnaissance scripts developed in this laboratory were executed exclusively within an isolated, authorized educational sandbox. No real-world targets, employee personal storage media, or unapproved production endpoints were engaged during this research.